Cloud Specialist [D68444-15-2025-23]

Unità Formativa : Programmazione Python

Docente: Massimo PAPA

Titolo argomento: Soluzione esercizi sulle stringhe e sulle liste

Es. sulle stringhe e sulle liste

Indicazioni sulla consegna **3**
Struttura del codice sorgente 4
Esercizi sulle stringhe **6**
Primo Esercizio 6
Secondo EsPrimo Esercizioercizio 6
Esercizi sulle liste **7**
Primo Esercizio 7
Secondo Esercizio 7
Terzo esercizio 7
Quarto esercizio 7
Soluzioni esercizi sulle stringhe **8**
Primo esercizio 8
Secondo esercizio 9
Soluzioni esercizi sulle liste **12**
Primo esercizio 12
Secondo esercizio 13
Terzo esercizio 14
Quarto esercizio 15

Indicazioni sulla consegna

Implementare gli algoritmi risolutivi dei seguenti esercizi codificandoli in linguaggio python. Partire analizzando il problema seguendo la seguente traccia:

- Quali sono gli input del problema?
- Quali sono gli output?
- Suddividi il problema in problemi più semplici e se lo ritieni opportuno descrivi la soluzione di ogni sottoproblema con un flow-chart
- Andrai a rappresentare ogni algoritmo che risolve un sottoproblema come una funzione
- La funzione restituisce un valore? Se si di che tipo?
- Ogni funzione accetta una lista di parametri formali? Se si di che tipo?
- Esegui la codifica partendo dal programma principale, al suo interno vai a richiamare le funzioni che poi andrai successivamente a definire.
- Continua la codifica dichiarando e definendo tutte le funzioni prima del programma principale che richiama tutte le altre. In testa alle funzioni scrivi un commento che descrive la funzione stessa , cosa restituisce e quali sono i parametri formali.
- Scrivi la sezione dell'inizializzazione variabili scrivendo in un commento il significa della variabile stessa
- Verifica la codifica utilizzando input di test, cercando di provare anche i casi limite.

Attenzione: non utilizzare strutture complesse oltre a stringhe e liste

Struttura del codice sorgente

Relativamente alle indicazioni di scrittura del codice, utilizza il seguente schema generale:

```
'''
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Testo esercizio

'''

##
## Funzioni:
##


def fun(param1: int, param2: float) -> int:
  '''
     Funzione: fun
      Template per costruire le funzioni
      Parametri formali:
          int Param1 -> descrizione Param1
          float Param2 -> descrizione Param2
      Valore di ritorno:
          int -> descrizione valore di ritorno
  '''
  retValue = 5 # Valore di ritorno della funzione

  return retValue


'''
    Programma principale
    Descrizione sintetica funzionalità
    del programma principale.
'''

# Sezione di input Dati

# Inizializzazioni variabili

# Elaborazione
```

```
# Eventuali sotto processi di Elaborazione

# Sezione di output
```

Esercizi sulle stringhe

Primo Esercizio

Scrivere un programma per rimuovere l'n- esimo carattere da una stringa non vuota. Progettare una funzione che accetti la stringa, la posizione del carattere e restituisca la stringa modificata.

Secondo Esercizio

Scrivere un programma che dica se una stringa è palindroma.

Esempio:

```
str="ABBA" PALINDROMA
str="PIPPO" NON PALINDROMA
```

Esercizi sulle liste

Primo Esercizio

Scrivere un programma python che rimuova gli elementi duplicati da una lista.

Esempio:

```
listaIN = [2, -4 ,5,6,5,5,2]
listaOUT=[2,-4,5,6]
```

Secondo Esercizio

Scrivere un programma che date due liste stampi "OK" se hanno almeno un membro comune altrimenti stampi "KO".

Esempio:

- `lista1=[1,5,8] lista2=[3,1,10]  -> output: "OK"`
- `lista1=[1,5,8] lista2=[3,11,10] -> output: "KO"`

Terzo esercizio

Scrivi un programma per trovare il secondo numero più piccolo in una lista.

Quarto esercizio

Scrivere un programma Python per dividere una data lista in due parti in cui viene data la lunghezza della prima parte della lista.

Esempio:

```
Lista originale: [1, 1, 2, 3, 4, 4, 5, 1]
Lunghezza della prima parte della lista: 3
Output : Prima parte: [1, 1, 2] ,
Seconda parte: [3, 4, 4, 5, 1]
```

Soluzioni esercizi sulle stringhe

Primo esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Primo Esercizio
    Scrivere un programma per rimuovere l'n- esimo
    carattere da una stringa non vuota. Progettare
    una funzione che accetti la stringa, la posizione
    del carattere e restituisca la stringa modificata.

"""


def rimuoviCarattere(stringa: str, n: int) -> str:
    """
    Funzione: rimuoviCarattere
    Permette di rimuovere l'ennesimo carattere dalla
    stringa in input alla funzione
    Parametri formali:
        str -> stringa da modificare
        n -> posizione carattere in stringa da rimuovere
    Valore di ritorno:
        str -> stringa modificata
    """

    # porzione stringa che precede carattere n
    primaParte = stringa[:n]  # porzione stringa che precede carattere n
    # porzione stringa che succede a carattere n
    if n == -1:  # gestisco il caso particolare
        secondaParte = ""
    else:
        secondaParte = stringa[n + 1 :]
    return primaParte + secondaParte


"""
Programma principale
Si chiede in input una stringa e una posizione al suo interno
In output si vuole la stringa senza l'ennesimo carattere
```

```
"""

# Sezione di input
stringa, pos = input("Inserisci la stringa e posizione: ").split()
pos = int(pos)

# Elaborazione
stringa = rimuoviCarattere(stringa, pos)

# Sezione di output
print(stringa)
```

Secondo esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo esercizio
    Scrivere un programma che dica se una stringa
    è palindroma.
    Esempio:
        str="ABBA" PALINDROMA
        str="PIPPO" NON PALINDROMA

"""


##
## Funzioni:
##


def inserisciStringa() -> str:
    """
    Funzione: inserisciStringa
     Permette all'utente di inserire una stringa
     dallo standard input
     Parametri formali:
         None
     Valore di ritorno:
         str stringa -> stringa inserita dall'utente
```

```
    """
    # inizializzazione stringa fuori dominio
    stringa = ""

    # loop di inserimento e controllo stringa
    while stringa == "":
        stringa = input("Inserisci la stringa (non vuota):")

    return stringa

def isPalindroma(stringa: str) -> bool:
    """
    Funzione: isPalindroma
    Verifica che la stringa passata come argomento formale
    sia palindromo o meno.
    Parametri formali:
        str stringa -> stringa su cui si esegue il controllo
    Valore di ritorno:
        bool ret -> set True allora palindorma altrimenti False
    """
    # inizializzazione valore di ritorno
    ret = True

    # Lunghezza della stringa
    lunStr = len(stringa)

    # loop di verifica
    pos=0
    while pos < lunStr / 2 and ret:
        if stringa[pos] != stringa[lunStr - pos - 1]:
            ret = False
        pos +=1

    return ret


def isPalindroma_FOR(stringa: str) -> bool:
    """
    Funzione: isPalindroma
    Verifica che la stringa passata come argomento formale
    sia palindromo o meno.
    Parametri formali:
        str stringa -> stringa su cui si esegue il controllo
    Valore di ritorno:
        bool ret -> set True allora palindorma altrimenti False
```

```
    """
    # inizializzazione valore di ritoeno
    ret = True

    # Lunghezza della stringa
    lunStr = len(stringa)

    # loop di verifica
    for pos in range(int(lunStr / 2)):
        if stringa[pos] != stringa[lunStr - pos - 1]:
            ret = False
            break

    return ret

def isPalindroma_PROPSTR(stringa: str) -> bool:
    """
    Funzione: isPalindroma
    Verifica che la stringa passata come argomento formale
    sia palindromo o meno.
    Parametri formali:
        str stringa -> stringa su cui si esegue il controllo
    Valore di ritorno:
        bool ret -> set True allora palindorma altrimenti False
    """

    return stringa == stringa[::-1]


"""
    Programma principale
    L'algoritmo stampa sullo standard output
    se la stringa inserita dall'utente è
    palindroma oppure no.
"""

# Sezione di input Dati
stringa = inserisciStringa()

# Elaborazione e output
if isPalindroma(stringa):
    print(stringa, "-> PALINDROMA")
else:
    print(stringa, "-> NON PALINDROMA")
```

Soluzioni esercizi sulle liste

Primo esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Primo esercizio
    Scrivere un programma python che rimuova gli elementi
    duplicati da una lista.
    Esempio:
        listaIN = [2, -4 ,5,6,5,5,2]
        listaOUT = [2,-4,5,6]

"""


def rimuoviDuplicati(lista: list) -> list:
    """
    Funzione: rimuoviDuplicati
    Permette di rimuovere tutti gli elementi
    duplicati presenti nella lista di ingresso.
    Parametri formali:
        list -> lista di input
    Valore di ritorno:
        list -> lista senza caratteri duplicati
    """
    listaOUT = []

    # loop di scansione lista di input
    for ele in listaIN:
        if ele not in listaOUT:
            # aggiungo elemento se non esiste in listaOUT
            listaOUT+=[ele]

    return listaOUT


"""
Programma principale
```

```
Si prova la funzione che rimuove gli elementi
duplicati all'interno di una lista.
"""

# Sezione di input (hard-coded)
listaIN = [1,1,3,4,5,5,6,7,7]

# Elaborazione
listaOUT = rimuoviDuplicati(listaIN)

# Sezione di output
print(listaOUT)
```

Secondo esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Secondo esercizio
    Scrivere un programma che date due liste stampi
    "OK"  se hanno almeno un membro comune altrimenti
    stampa "KO".
    Esempio:
        lista1=[1,5,8] lista2=[3,1,10]  -> output: "OK"
        lista1=[1,5,8] lista2=[3,11,10] -> output: "KO"

"""


def hasElementoComune(lista1: list, lista2: list) -> bool:
    """
    Funzione: hasElementoComune
    Verifica che esista almeno un elemento
    in comune alle liste date in input.
    Se si allora si restituisce True
    False altrimenti.
    Parametri formali:
        list -> lista1
        list -> lista2
    Valore di ritorno:
```

```
        bool -> True se elemento comune altrimenti False
    """
    # flag valore di ritorno della funzione
    flagComune = False

    # loop principale di elaborazione
    for ele in lista1:
        if ele in lista2:  # verifica che ele sia comune
            flagComune = True
            break

    return flagComune


"""
Programma principale
Si prova la funzione che verifica se tra due
liste esiste almeno un elemento in comune.
"""

# Sezione di input (hard-coded)
lista1 = [1, 5, 8]
lista2 = [3, 1, 10]

# Elaborazione e output
if hasElementoComune(lista1, lista2):
    print("OK")
else:
    print("KO")
```

Terzo esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Terzo esercizio
    Scrivi un programma per trovare il secondo numero
    più piccolo in una lista.
```

```
"""

# Sezione di input (hard-coded)
lista = [1, 2, 5, 7, 8]

# Ordina la lista in ordine crescente
lista.sort(reverse=False)

# Stampa il secondo elemento che
# rappresenta il secondo più piccolo

# Se esistono elementi duplicati
if lista[0:1]==lista[1:2] :
    # rimuovo gli elementi duplicati in testa
    lista.remove(lista[0:1][0])
    while(lista[0:1]==lista[1:2]):
        lista.remove(lista[0:1][0])

# stampo il secondo elemento piu' piccolo
print(lista[1:2])
```

Quarto esercizio

```
"""
    Autore:  Nome Cognome
    Data: gg/mm/aaaa

    Titolo: Quarto esercizio
    Scrivere un programma Python per dividere una data
    lista in due parti in cui viene data la lunghezza
    della prima parte della lista.
    Esempio:
        Lista originale: [1, 1, 2, 3, 4, 4, 5, 1]
        Lunghezza della prima parte della lista: 3
        Output :      Prima parte: [1, 1, 2] ,
                    Seconda parte: [3, 4, 4, 5, 1]

"""
```

```
def rimuoviElemento(lista: list, n: int) -> list:
    """
    Funzione: rimuoviElemento
    Permette di rimuovere l'ennesimo Elemento dalla
    stringa in input alla funzione
    Parametri formali:
        list -> lista da modificare
        n -> posizione elemento nella lista da rimuovere
    Valore di ritorno:
        list -> lista modificata
    """
    # porzione lista che precede elemento n
    primaParte = lista[0:n]  # porzione lista che precede elemento n
    # porzione lista che succede a elemento n
    if n == -1:  # gestisco il caso particolare
        secondaParte = []  # secondaParte=[n:0]
    else:
        secondaParte = lista[n + 1 :]
    return primaParte + secondaParte


def spezzaLista(lista: list, lunghezza: int) -> tuple:
    """
    Funzione: spezzaLista
    Permette di "spezzare" la lista in input
    in due liste
    Parametri formali:
        list -> lista da modificare
        int -> lunghezza prima lista
    Valore di ritorno:
        int,list,list -> lista1 e lista2 tronconi di lista in input
                         errore: se False allora no errore
                         altrimenti True problema di lunghezza o lista
                         vuota
    """

    # inizializzazione variabili
    lista1 = []
    lista2 = []
    errore = True

    # verifica situazione di errore
    if 0 <= lunghezza < len(lista) and len(lista) != 0:  # no errore
allora procedo
        lista1 = lista[:lunghezza]
        lista2 = lista[lunghezza:]
```

```
        errore = False

    return errore, lista1, lista2


"""
Programma principale
Si chiede in input una lista (input hard-coded) e la lunghezza
della prima lista
In output si vogliono due liste
"""

# Sezione di input
# stringa, pos = input("Inserisci la stringa e posizione: ").split()
# pos = int(pos)

lista = [1, 2, 3, 4, 5, 6]
lun = len(lista) - 1
# Elaborazione
# lista = rimuoviElemento(lista, pos)
err, lista1, lista2 = spezzaLista(lista, lun)

# Sezione di output
if not err:
    print(lista1, lista2)
else:
    print("ERRORE!!")
```